Dynamic Programming I

Memoization, tabulation & 1D DP

Learning goals

- Recognise **overlapping subproblems** and **optimal substructure**
- Follow the recipe: **state → transition → base case → order → answer**
- Write DP **top-down** (memoization) and **bottom-up** (tabulation)
- Count with DP (number of ways) and optimise with DP (min/max)
- Add a small extra dimension to the state ("what did I do last?")
- Reconstruct an optimal solution, and save memory with rolling arrays
- Complexity = **#states × cost per transition**

Motivation: AtCoder DP A — Frog 1

Stones 1..n with heights h_i. A frog on stone i jumps to i+1 or i+2, paying $|h_i - h_j|$. Minimum total cost from stone 1 to stone n?

```
h:    10  30  40  20
best: 1 → 2 → 4   cost 20 + 10 = 30
```

Brute force over jump sequences: ~φ^n paths (Fibonacci many). ×

But "cheapest way to reach stone i" doesn't depend on how we continue...

From recursion to DP: Fibonacci

```
long long fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

```
fib(5) calls fib(3) twice, fib(2) three times, …
fib(40): 331,160,281 calls — but only 41 distinct arguments!
```

Overlapping subproblems: the same call is solved again and again.

Fix: **remember** each answer the first time → O(n).

Memoization (top-down)

```
vector<long long> memo(N, -1);          // -1 = "not computed"
long long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
```

- Same recursion, plus a cache lookup
- Each state computed **once** → O(n) total
- Only reachable states are computed
- Needs a sentinel that can't be a real answer (or a `bool` array)

Tabulation (bottom-up)

```
vector<long long> f(n + 1);
f[0] = 0; f[1] = 1;
for (int i = 2; i <= n; i++) f[i] = f[i - 1] + f[i - 2];
```

- Fill a table in an order where dependencies are **already filled**
- No recursion → no stack overflow, smaller constant
- Easier to optimise memory (keep only last two values)

Same states, same transitions — different evaluation order.

The DP recipe

1. **State**: what does `dp[i]` *mean*? Write it in one sentence.
2. **Transition**: how is `dp[i]` built from smaller states?
   (Think about the **last decision**.)
3. **Base case(s)**: smallest states, answered directly
4. **Order**: compute dependencies first
5. **Answer**: which state(s) hold the final result?

Complexity = (number of states) × (work per transition)

Frog 1, by the recipe

1. **State**: `dp[i]` = min cost to reach stone i
2. **Transition** — last jump came from i−1 or i−2:
   `dp[i] = min(dp[i-1] + |h[i]-h[i-1]|, dp[i-2] + |h[i]-h[i-2]|)`
3. **Base**: `dp[0] = 0`
4. **Order**: i increasing
5. **Answer**: `dp[n-1]`

```
h:   10  30  40  20
dp:   0  20  30  30
```

O(n) states × O(1) → **O(n)**.

Frog 1: both styles

```cpp
// top-down
long long solve(int i) {
    if (i == 0) return 0;
    long long &res = memo[i];
    if (res != -1) return res;
    res = solve(i - 1) + abs(h[i] - h[i - 1]);
    if (i >= 2) res = min(res, solve(i - 2) + abs(h[i] - h[i - 2]));
    return res;
}
// bottom-up
dp[0] = 0;
for (int i = 1; i < n; i++) {
    dp[i] = dp[i - 1] + abs(h[i] - h[i - 1]);
    if (i >= 2) dp[i] = min(dp[i], dp[i - 2] + abs(h[i] - h[i - 2]));
}
```

Why greedy fails here

Greedy: "always jump to the cheaper next stone".

```
h:  30  10  60  10  60  50
```

Greedy: 30→10 (20), 10→10 via i+2 (0), 10→50 via i+2 (40) = **60**
DP finds **40**: 1 → 3 → 5 → 6 (30 + 0 + 10).

DP considers **all** choices, but shares work between them.
Optimal substructure: the best path to i ends with a best path to i−1 or i−2.

Counting with DP: CSES 1633 Dice Combinations

Number of ordered sequences of throws (1..6) summing to n, mod 10^9+7.

- **State**: `dp[s]` = # sequences with sum s
- **Last throw** was d ∈ 1..6: `dp[s] = Σ dp[s − d]`
- **Base**: `dp[0] = 1` (the empty sequence)

```
s:   0  1  2  3  4  5  6   7
dp:  1  1  2  4  8 16 32  63
```

n = 3 → 4: (1,1,1) (1,2) (2,1) (3). O(6n).

Counting rules

- Sum over **disjoint** cases (classify by the last decision)
- Every object must be counted **exactly once**
- Base case for "empty" is usually **1**, not 0
- Take the modulus at every addition (Week 11)

```
for (int s = 1; s <= n; s++)
    for (int d = 1; d <= 6 && d <= s; d++)
        dp[s] = (dp[s] + dp[s - d]) % MOD;
```

Optimisation: CSES 1634 Minimizing Coins

Coins $c_1..c_n$ (unlimited). Fewest coins with total x, or −1.

- **State**: `dp[s]` = fewest coins summing to s (∞ if impossible)
- **Transition**: `dp[s] = min over coins c ≤ s of dp[s − c] + 1`
- **Base**: `dp[0] = 0`
- **Answer**: `dp[x]`, or −1 if ∞

$O(n \cdot x) = 100 \cdot 10^6 = 10^8$ simple ops — fine.

Minimizing Coins: trace

coins {1, 5, 7}, x = 11

```
s:   0 1 2 3 4 5 6 7 8 9 10 11
dp:  0 1 2 3 4 1 2 1 2 3  2  3
```

dp[11] = min(dp[10], dp[6], dp[4]) + 1 = min(2, 2, 4) + 1 = **3** (5 + 5 + 1)

Greedy (largest coin first): 7 + 1 + 1 + 1 + 1 = 5 coins ×

Reconstructing the solution

Store **which choice** achieved the optimum:

```
for (int s = 1; s <= x; s++)
    for (int v : c)
        if (v <= s && dp[s - v] + 1 < dp[s]) {
            dp[s] = dp[s - v] + 1;
            last[s] = v;                    // remember the decision
        }
for (int s = x; s > 0; s -= last[s]) print(last[s]);   // 1 5 5
```

Alternatively, re-derive: find any c with `dp[s − c] + 1 == dp[s]`.

Order matters: Coin Combinations I vs II

coins {2, 3, 5}, x = 9

	Counts	Loops	Result
CSES 1635	**ordered** sequences (2+3+2+2 ≠ 2+2+2+3)	sum outer, coin inner	8
CSES 1636	**multisets**	coin outer, sum inner	3

Coin-outer: each multiset is built in coin order → counted once.

(Full treatment of knapsack-style DP: **Week 15**.)

Adding a dimension: AtCoder DP C — Vacation

Each day choose activity A/B/C with happiness a_i, b_i, c_i.
Never the same activity on two consecutive days. Maximise.

- `dp[i]` alone is not enough: we must know **what we did yesterday**
- **State**: `dp[i][j]` = best total for days 0..i, doing j on day i
- `dp[i][j] = a[i][j] + max over k ≠ j of dp[i−1][k]`
- $O(3 \cdot 3 \cdot n)$

Rule: if the transition needs information, **put it in the state**.

Rolling arrays

`dp[i]` depends only on `dp[i−1]` (and maybe `dp[i−2]`)?

Keep just those rows:

```
array<long long, 3> dp = a[0];
for (int i = 1; i < n; i++) {
    array<long long, 3> nd{};
    for (int j = 0; j < 3; j++) {
        long long best = 0;
        for (int k = 0; k < 3; k++) if (k != j) best = max(best, dp[k]);
        nd[j] = best + a[i][j];
    }
    dp = nd;
}
```

Memory O(n) → **O(1)**. (You lose reconstruction.)

Top-down vs bottom-up

	Memoization	Tabulation
Writing	mirrors the recurrence	must choose an order
States computed	only reachable ones	all of them
Overhead	recursion, stack depth	tight loops
Memory tricks	hard	rolling arrays easy
Risk	stack overflow (depth ~10^6)	wrong order

Start with whichever you find clearer; convert when needed.

Push vs pull

Pull: `dp[i]` collects from its predecessors (what we've done).

Push: `dp[i]` sends its value forward to successors:

```
for (int i = 0; i < n; i++) {
    if (i + 1 < n) dp[i+1] = min(dp[i+1], dp[i] + cost(i, i+1));
    if (i + 2 < n) dp[i+2] = min(dp[i+2], dp[i] + cost(i, i+2));
}
```

Same complexity; push is handy when "where can I go from here" is easier to describe than "where could I have come from".

Pitfalls

- State not defined precisely → contradictory transitions
- Missing information in the state (Vacation without "last activity")
- Wrong base case: counting usually needs `dp[0] = 1`
- Using ∞ = `INT_MAX` then adding 1 → **overflow** to negative
- Memo sentinel −1 that is also a valid answer
- Evaluating states before their dependencies (bottom-up order)
- Forgetting the modulus in counting DPs
- Deep recursion in memoization for n ~ 10^6+

Practice problems

- AtCoder DP A — Frog 1 · B — Frog 2 · C — Vacation
- CSES 1633 — Dice Combinations
- CSES 1634 — Minimizing Coins
- CSES 1635 / 1636 — Coin Combinations I / II
- CSES 1637 — Removing Digits
- Codeforces 189A — Cut Ribbon
- Codeforces 455A — Boredom
- Codeforces 698A — Vacations
- Codeforces 166E — Tetrahedron

Summary

- DP = recursion over **states** + **reuse** of answers
- Recipe: state, transition (last decision), base, order, answer
- Memoization and tabulation compute the same table
- Counting: sum disjoint cases; optimising: min/max over choices
- Missing information? Add a dimension to the state
- Time = states × transitions; memory often reducible by rolling
- Next week: **DP II** — knapsack, LIS, LCS, edit distance, grid DP